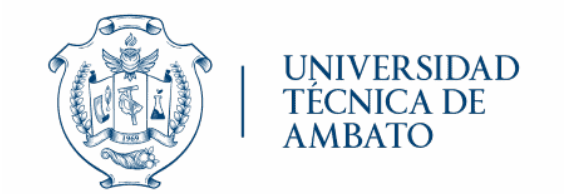
UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL
CARRERA DE INGENIERÍA DE SOFTWARE



CATÁLOGO DE LA BIBLIOTECA FISEI

Árbol general y árbol binario de búsqueda en Java

Asignatura	Estructura de Datos
Nivel	Tercero – Ingeniería de Software
Modalidad	Individual
Estudiante	Fernando Villacres
Docente	José Caiza
Fecha	22 de octubre de 2026

Informe de práctica

1. Presentación

En esta práctica se construyó una aplicación de consola para representar la organización de la Biblioteca FISEI y consultar sus libros. La biblioteca se organiza en áreas y estantes; dentro de cada estante, los libros se ordenan por su código. Para representar estas dos necesidades se implementaron un árbol general y un árbol binario de búsqueda, respectivamente.

Además de las operaciones básicas, el trabajo permite ver paso a paso los recorridos recursivos. Esto ayuda a relacionar el orden en que se ejecutan las llamadas con el resultado que aparece en pantalla.

2. Cómo se organizó el programa

El código se separó en paquetes para distinguir los tipos de unidad, las clases del modelo, la lógica de los árboles y el programa de consola. Cada estante crea su propio catálogo; las áreas y la biblioteca, en cambio, solo mantienen a sus hijos.

Paquete o clase	Función dentro del programa
enums	Define los tipos de unidad y de recorrido.
modelo.Libro	Guarda el código, título, autor y año, aplicando las validaciones.
modelo.UnidadBiblioteca	Representa una biblioteca, área o estante.
negocio.ArbolBiblioteca	Administra la jerarquía y sus consultas.
negocio.CatalogoABB	Administra los libros de un estante mediante un ABB.
app.Main y DatosPrueba	Presentan el menú y cargan los datos de ejemplo.
app.PruebasBiblioteca	Comprueba los casos de la consigna.

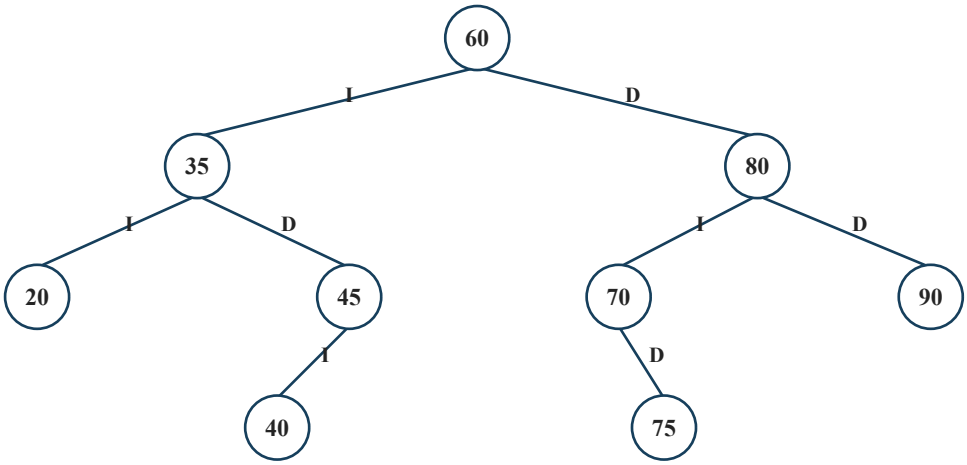
El código de un libro debe ser positivo y no se puede cambiar después de creado. El título y el autor no pueden estar vacíos, y el año debe estar entre 1900 y 2026. Los códigos de las unidades se guardan en mayúsculas, así que una búsqueda como “sw” encuentra la unidad “SW”.

3. La biblioteca de ejemplo

```
BIB Biblioteca FISEI
  SW Software
    SW-E1 Estante 1 de Software: 60, 35, 80, 20, 45, 70, 90, 40, 75
    SW-E2 Estante 2 de Software: 15, 10, 25
  TI Tecnologías de la Información
    TI-E1 Estante 1 de TI: 50
  TEL Telecomunicaciones
```

La estructura tiene siete unidades, cuatro hojas y una altura de dos aristas. Entre los tres estantes hay trece libros. En el ABB de SW-E1, por ejemplo, 35 queda a la izquierda de 60 porque su código es menor; 80 queda a la derecha porque es mayor.

Árbol inicial del estante SW-E1 (I: hijo izquierdo, D: hijo derecho)


60
I
D
35
80
I
D
I
D
20
45
70
90
I
D
40
75

4. Recorridos y consultas

Los tres recorridos visitan los mismos nueve libros, pero cambian el momento en que se procesa cada nodo. En el inorden, al respetarse el orden del ABB, los códigos aparecen de menor a mayor.

Operación	Resultado
Preorden	60 35 20 45 40 80 70 75 90
Inorden	20 35 40 45 60 70 75 80 90
Posorden	20 40 45 35 75 70 90 80 60
Búsqueda de 42	Camino visitado: [60, 35, 45, 40]
Rango de 40 a 75	5 libros: 40, 45, 60, 70 y 75

La traza escribe “entra” cuando llega a un nodo, “VISITA” cuando le corresponde procesarlo y “null → regresa” al alcanzar un hijo vacío. La sangría permite reconocer la profundidad de cada llamada.

Fragmento de la traza de preorden

```
entra 60
VISITA 60
  entra 35
  VISITA 35
    entra 20
    VISITA 20
      null -> regresa
      null -> regresa
    entra 45
    VISITA 45
      entra 40
      VISITA 40
        null -> regresa
        null -> regresa
      null -> regresa
  entra 80
  VISITA 80
  ...
Secuencia PREORDEN: 60 35 20 45 40 80 70 75 90
```

La traza completa de los tres órdenes se puede generar desde la opción correspondiente del menú, seleccionando el estante SW-E1.

5. Inserción y eliminación en el ABB

El catálogo rechaza un código duplicado. También se probaron las tres situaciones de eliminación: quitar una hoja, quitar un nodo con un solo hijo y quitar uno con dos hijos. En el último caso se reemplaza el dato por el sucesor inorden, es decir, el menor valor del subárbol derecho; así se conserva el orden del ABB.

Para el caso indicado en la consigna, se inserta primero 42 y luego se eliminan 20, 70 y 60, en ese orden. El preorden final obtenido es 75 35 45 40 42 80 90.

6. Pruebas

Se compiló con OpenJDK 17 y se ejecutó la clase PruebasBiblioteca. La ejecución local terminó con 12/12 pruebas OK. Las pruebas incluyen la biblioteca vacía, los datos iniciales, niveles y búsquedas, reglas para agregar unidades, recorridos, duplicados, rango, camino de búsqueda, modificación, los casos de eliminación y la reconstrucción opcional desde preorden.

```
OK  biblioteca vacía
```

```
OK  estructura inicial
OK  niveles y búsqueda sin distinguir mayúsculas
OK  reglas de inserción y códigos únicos
OK  recorridos del estante SW-E1
OK  duplicado, camino y rango
OK  eliminación de hoja, un hijo y dos hijos
OK  modificación conserva código y valida cambios
OK  operaciones sobre libro inexistente
OK  validaciones de datos de libro
OK  ABB inválido por nieto mal ubicado
OK  reconstrucción directa desde preorden
12/12 pruebas OK
```

7. Algunas ideas que deja el ejercicio

¿Por qué la búsqueda en el ABB es O(h), mientras que en el árbol general es O(N)?

En el ABB, cada comparación decide si continuar por la izquierda o por la derecha. Por eso se sigue un solo camino, cuya longitud depende de la altura h. En el árbol general no se compara siguiendo un orden de claves; en el peor caso hay que revisar todas las N unidades. Si los códigos se insertan ya ordenados en un ABB sin balanceo, el árbol puede quedar como una cadena. Entonces h llega a n−1 y la búsqueda deja de ser eficiente: pasa a O(n).

¿Por qué usar el sucesor inorden al eliminar un nodo con dos hijos?

El sucesor es el menor de los códigos que son mayores que el nodo que se elimina. Al usarlo, los valores del lado izquierdo siguen siendo menores y los del lado derecho siguen siendo mayores. Elegir cualquier otro valor del subárbol derecho podría romper ese orden.

¿Cómo podría servir algo parecido en SmartETL?

Un árbol general podría ayudar a mostrar una canalización ETL como una jerarquía de proyecto, fuentes de datos, etapas y tareas. Un ABB podría servir para consultar registros por una clave numérica y buscar rangos. Si los datos crecen mucho o pueden llegar ordenados, habría que considerar un árbol balanceado o un índice de base de datos para mantener tiempos razonables.

> *Estas respuestas resumen lo que se observa en la implementación. Conviene revisarlas y expresarlas con palabras propias antes de entregar el informe.*

8. Repositorio y estado de la entrega

El código fuente y el historial de cambios están en el repositorio público: github.com/fercho123909/Deber-Arboles-VillacresFernando. La implementación está en la rama desarrollo y hay un workflow de GitHub Actions preparado para compilar y ejecutar las pruebas.

La rama main todavía no contiene la implementación: falta crear e integrar el pull request desde desarrollo. Tampoco se incluyen capturas simuladas; las capturas de ejecución, historial y PR deben tomarse cuando se complete ese paso.

9. Antes de subirlo a Moodle

Quedan tres cosas personales por completar: agregar en TRAZAS.md los tres códigos obtenidos a partir de la regla del enunciado (sin publicar el número de cédula), crear e integrar el pull request y tomar las capturas reales solicitadas. El borrador de reflexión también debe revisarse para que represente la comprensión y experiencia del estudiante.

Para compilar, ejecutar la aplicación y repetir las pruebas, las instrucciones están en el README del repositorio.